

Theoretical Assignment – Question 6

Date/DateTime Functions, String Functions and Regular Expressions in PySpark

1. Date and DateTime Functions in PySpark

PySpark provides functions for working with dates and timestamps. They can extract date parts, format dates, calculate differences, add/subtract dates, find boundaries, and work with timestamps and time zones.

```
Date / DateTime Functions
■
■■■ Extract: year(), month(), day(), dayofmonth(),
    dayofweek(), dayofyear()
■
■■■ Format: date_format()
■■■ Differences: datediff(), months_between()
■■■ Add/subtract: add_months(), date_add(), date_sub()
■■■ Boundaries: last_day(), trunc()
■■■ Current/time zone: current_date(), current_timestamp(),
    from_utc_timestamp(), convert_timezone()
```

Extracting Parts of a Date

```
input_df = input_df.withColumn('year', year('order_date')) \
    .withColumn('month', month('order_date')) \
    .withColumn('day', day('order_date')) \
    .withColumn('dayofmonth', dayofmonth('order_date')) \
    .withColumn('day_of_week', dayofweek('order_date')) \
    .withColumn('day_of_year', dayofyear('order_date'))
```

For an order_date such as 2026-09-20: year() gives 2026, month() gives 9, and day()/dayofmonth() give 20. dayofweek() gives the weekday number and dayofyear() gives the position within the year.

Remember: day() / dayofmonth() → day of month; dayofweek() → weekday; dayofyear() → position in year.

Formatting Dates with date_format()

date_format() converts a date/timestamp into a formatted string using the supplied pattern.

```
input_df = input_df.withColumn('year_month', date_format('order_date', 'yyyy-MM')) \
    .withColumn('month_date', date_format('order_date', 'MM-dd')) \
    .withColumn('date_month_year', date_format('order_date', 'dd-MM-yyyy')) \
    .withColumn('date_month_year1', date_format('order_date', 'dd/MM/yyyy')) \
    .withColumn('date_month_year2', date_format('order_date', 'dd.MM.yyyy')) \
    .withColumn('year_month1', date_format('order_date', 'yyyy-MMM')) \
    .withColumn('year_month2', date_format('order_date', 'yyyy-MMMM')) \
    .withColumn('month_year', date_format('order_date', 'MMMM yyyy'))
```

For 2026-09-20, these produce values such as 2026-09, 09-20, 20-09-2026, 20/09/2026, 20.09.2026, 2026-Sep, 2026-September and September 2026.

Date Differences

```
input_df.withColumn('days_bw_order', datediff(current_date(), 'order_date')) \
    .withColumn('months_bw_order', months_between(current_date(), 'order_date')) \
    .withColumn('months_bw_order1', floor(months_between(current_date(), 'order_date'))) \
    .withColumn('months_bw_order2', ceil(months_between(current_date(), 'order_date'))) \
    .withColumn('months_bw_order_cast', months_between(current_date(), 'order_date').cast('int')) \
    .withColumn('months_bw_order_round', round(months_between(current_date(), 'order_date'), 2))
```

datediff() returns the difference in days. **months_between()** returns the difference in months and can contain decimals. floor() rounds down, ceil() rounds up, cast('int') converts to an integer, and round(..., 2) rounds to two decimal places.

```
5.67
↓ floor() → 5
↓ ceil() → 6
↓ round(..., 2) → 5.67
```

Adding and Subtracting Dates

```
df.withColumn('future_month', add_months('joining_date', 4)) \
    .withColumn('past_month', add_months('joining_date', -4)) \
    .withColumn('future_date', date_add('joining_date', 31)) \
    .withColumn('past_date', date_add('joining_date', -31)) \
    .withColumn('date_sub', date_sub('joining_date', 31))
```

add_months() moves by months. date_add() moves by days. date_sub() subtracts days.

Month and Year Boundaries

```
df.withColumn('month_end', last_day('joining_date')) \
  .withColumn('month_start', trunc('joining_date', 'month')) \
  .withColumn('year_start', trunc('joining_date', 'year'))
```

`last_day()` returns the last day of the month. `trunc(..., 'month')` gives the first day of that month, while `trunc(..., 'year')` gives the first day of that year.

```
2026-09-20
↓ trunc(..., 'month') → 2026-09-01
↓ trunc(..., 'year') → 2026-01-01
```

next_day()

```
df.withColumn('next_monday', next_day('joining_date', 'Saturday'))
```

`next_day(date, weekday)` finds the next occurrence of the specified weekday after the given date.

Your classroom code names the column `next_monday` but asks for `Saturday`. The function is searching for `Saturday`; if `Monday` is intended, use `'Monday'`.

Current Date, Timestamp and Time Zones

```
df.withColumn('current_timestamp', current_timestamp()) \
  .withColumn('current_timestamp_india',
    from_utc_timestamp(current_timestamp(), 'Asia/Kolkata')) \
  .withColumn('current_timestamp_india1',
    convert_timezone(None, lit('Asia/Kolkata'), current_timestamp()))
```

`current_date()` returns the current date. `current_timestamp()` returns the current timestamp. The time-zone functions convert a timestamp to a specified time zone such as `Asia/Kolkata`.

```
UTC timestamp
↓
Time-zone conversion
↓
Asia/Kolkata timestamp
```

2. String Functions in PySpark

String functions operate on text values. In your example, `emp_name` may contain values such as `'Abhishek Kumar'`.

String Functions

```
■
■■■ split()
■■■ upper()
■■■ lower()
■■■ ltrim()
■■■ rtrim()
■■■ trim()
■■■ length()
```

split()

```
df.withColumn('names', split('emp_name', ' ')) \
  .withColumn('first_name', split('emp_name', ' ')[0]) \
  .withColumn('last_names', split('emp_name', ' ')[1])
```

`split()` divides a string using a delimiter and returns an array. For `'Abhishek Kumar'`, `split` on a space produces `['Abhishek', 'Kumar']`; index `[0]` is the first element and `[1]` is the second.

upper() and lower()

```
df.withColumn('upper_name', upper('emp_name')) \
  .withColumn('lower_name', lower('emp_name'))
```

`upper()` converts text to uppercase. `lower()` converts text to lowercase.

ltrim(), rtrim(), trim() and length()

```
df.withColumn('ltrim', ltrim('emp_name')) \
  .withColumn('rtrim', rtrim('emp_name')) \
  .withColumn('trim', trim('emp_name')) \
  .withColumn('length', length('emp_name'))
```

`ltrim()` removes whitespace from the left side, `rtrim()` removes whitespace from the right side, `trim()` removes whitespace from both sides, and `length()` returns the number of characters.

Complete String Example

```
df.withColumn('names', split('emp_name', ' ')) \
.withColumn('first_name', split('emp_name', ' ')[0]) \
.withColumn('last_names', split('emp_name', ' ')[1]) \
.withColumn('upper_name', upper('emp_name')) \
.withColumn('lower_name', lower('emp_name')) \
.withColumn('ltrim', ltrim('emp_name')) \
.withColumn('rtrim', rtrim('emp_name')) \
.withColumn('trim', trim('emp_name')) \
.withColumn('length', length('emp_name')) \
.show()

emp_name
■
■■■ split() → Array → [0] first_name / [1] last_name
■■■ upper() → Uppercase
■■■ lower() → Lowercase
■■■ ltrim() → Remove left spaces
■■■ rtrim() → Remove right spaces
■■■ trim() → Remove both-side spaces
■■■ length() → Number of characters
```

3. Regular Expressions in PySpark

A **regular expression (Regex)** is a pattern used to find or extract specific text from a string.

regexp_extract() — Phone Number Example

```
df.withColumn(
    'phone_number',
    regexp_extract('number', r'(\d{10})$', 1)
).show()
```

The pattern **(\d{10})\$** means: find exactly 10 digits at the end of the string.

```
(      ) → Capturing group
\d      → A digit
{10}    → Exactly 10 digits
$       → End of the string
```

For example, if number contains 'Phone: 9876543210', the extracted value is 9876543210.

regexp_extract() — Order ID Example

```
df.withColumn(
    'order_id',
    regexp_extract('description', r'(ORD-\d+)', 1)
).show()
```

For a description such as 'Order successfully completed: ORD-12345', the pattern extracts ORD-12345.

```
(ORD-\d+)
■      ■
■      ■■■ one or more digits
■■■■■■ literal text ORD-
```

Understanding the 1 in regexp_extract()

```
regexp_extract(
    column,
    pattern,
    group_number
)
```

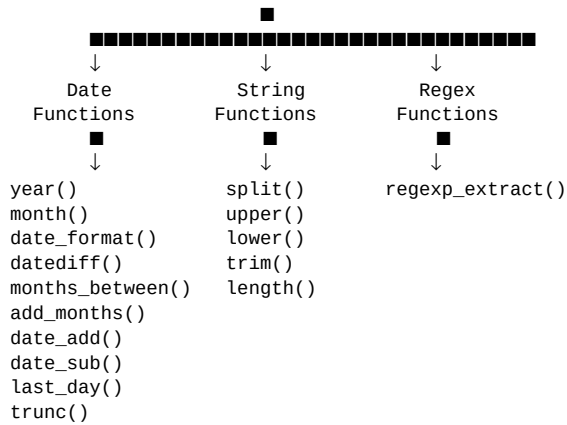
The 1 means: return the value captured by the first capturing group. In (ORD-\d+), the entire expression inside the parentheses is capturing group 1.

Common Regex Symbols Used Here

```
\d      → digit
\d+     → one or more digits
\d{10}  → exactly 10 digits
()      → capturing group
$       → end of string
ORD-    → literal text
```

Complete Mental Model

PySpark Functions



Quick Revision

DATE / DATETIME

year()	→ extract year
month()	→ extract month
day()/dayofmonth()	→ extract day
dayofweek()	→ weekday number
dayofyear()	→ day position in year
date_format()	→ format date as string
datediff()	→ difference in days
months_between()	→ difference in months
add_months()	→ add/subtract months
date_add()	→ add days
date_sub()	→ subtract days
last_day()	→ last day of month
trunc()	→ truncate to month/year
next_day()	→ next specified weekday
current_date()	→ current date
current_timestamp()	→ current timestamp

STRING

split()	→ split into an array
upper()	→ uppercase
lower()	→ lowercase
ltrim()	→ remove left spaces
rtrim()	→ remove right spaces
trim()	→ remove both-side spaces
length()	→ string length

REGEX

regexp_extract()	→ extract text matching a pattern
------------------	-----------------------------------